AIR UNIVERSITY ISLAMABAD

Department of Computer Science

Full Stack Web Development frontend lab
Lab Task # 06

Name: Roheela

Class: ADSCS-3-A

Reg ID: 2500627

Date: 10/20/26

Submitted To Raja Faisal Umar

LAB TASK 06
Topic: Node Js

Task 01: Notes Tool (Node.js + fs)
Create notes.js, a command-line tool that saves notes in a JSON file.
- Run it like this: node notes.js add "Buy milk", node notes.js list, node notes.js clear
- Read the command and text from process.argv.
- Save notes in notes.json with JSON.stringify and read them back with JSON.parse.
- If notes.json does not exist, start with an empty list. Show a clear message for a wrong command.

Notes.js code:

```
const fs = require('fs');
const FILE_PATH = 'notes.json';

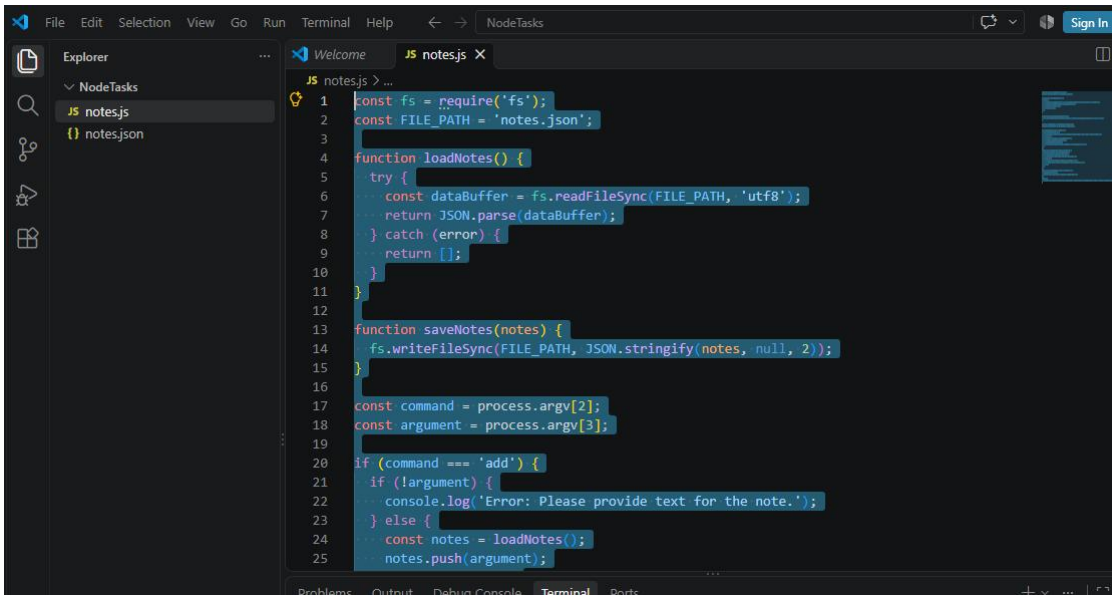
function loadNotes() {
  try {
    const dataBuffer = fs.readFileSync(FILE_PATH, 'utf8');
    return JSON.parse(dataBuffer);
  } catch (error) {
    return [];
  }
}
```

```
function saveNotes(notes) {
  fs.writeFileSync(FILE_PATH, JSON.stringify(notes, null, 2));
}
```

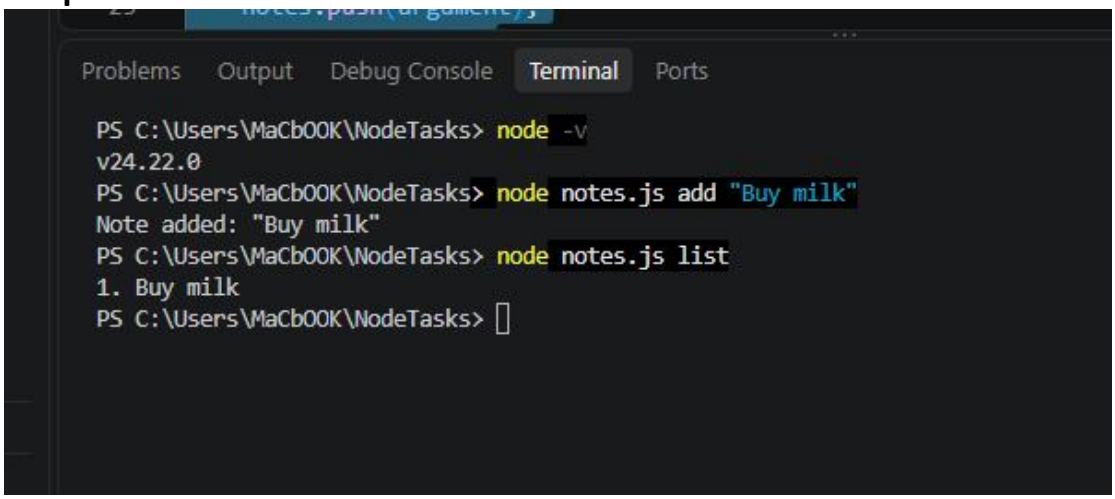
```
const command = process.argv[2];
const argument = process.argv[3];
```

```
if (command === 'add') {
  if (!argument) {
    console.log('Error: Please provide text for the note.');
  } else {
    const notes = loadNotes();
    notes.push(argument);
    saveNotes(notes);
    console.log(`Note added: "${argument}"`);
  }
}
else if (command === 'list') {
```

```
  const notes = loadNotes();
  if (notes.length === 0) {
    console.log('No notes found.');
  } else {
    notes.forEach((note, index) => {
      console.log(`${index + 1}. ${note}`);
    });
  }
}
else if (command === 'clear') {
  saveNotes([]);
  console.log('All notes cleared.');
}
else {
  console.log('Invalid command! Available commands: add, list, clear');
}
```



Output:



Task 2: Students API (Express + Postman)
- Create the project folder, run npm init -y and npm install express.
- Build GET /students and GET /students/:id (404 if not found).
- Build POST /students with validation (name, email must contain @, program). 400 for wrong data, 201 when created.
- Build PUT /students/:id and DELETE /students/:id.
- Save students in students.json with fs so data stays after restart.
- Test every route in Postman. Bonus: GET /students?program=BSCS.

Code:

```
const express = require('express');
const fs = require('fs');

const app = express();
const PORT = 3000;
const FILE_PATH = 'students.json';
```

```
app.use(express.json());
```

```
function loadStudents() {
  try {
    const dataBuffer = fs.readFileSync(FILE_PATH, 'utf8');
    return JSON.parse(dataBuffer);
  } catch (error) {
    return [];
  }
}
```

```
function saveStudents(students) {
  fs.writeFileSync(FILE_PATH, JSON.stringify(students, null, 2));
}
```

```
// 1. GET /students (Bonus: Filter by program e.g.
/students?program=BSCS)
app.get('/students', (req, res) => {
  let students = loadStudents();
  const { program } = req.query;
```

```
  if (program) {
    students = students.filter(s => s.program.toLowerCase() ===
program.toLowerCase());
  }
```

```
  res.status(200).json(students);
});
```

```
// 2. GET /students/:id
app.get('/students/:id', (req, res) => {
  const students = loadStudents();
  const studentId = parseInt(req.params.id);
  const student = students.find(s => s.id === studentId);
```

```
  if (!student) {
    return res.status(404).json({ error: 'Student not found' });
  }
```

```
  res.status(200).json(student);
});
```

```
// 3. POST /students (With validation)
app.post('/students', (req, res) => {
  const { name, email, program } = req.body;
```

```
  if (!name || typeof name !== 'string' || name.trim() === '') {
    return res.status(400).json({ error: 'Validation Error: Name is
required' });
  }
  if (!email || typeof email !== 'string' || !email.includes('@')) {
    return res.status(400).json({ error: 'Validation Error: Email must
contain @' });
  }
  if (!program || typeof program !== 'string' || program.trim() === ''
{
    return res.status(400).json({ error: 'Validation Error: Program is
required' });
  }
```

```
  const students = loadStudents();
  const newId = students.length > 0 ? students[students.length - 1].id
+ 1 : 1;
```

```
  const newStudent = { id: newId, name: name.trim(), email:
email.trim(), program: program.trim() };
  students.push(newStudent);
  saveStudents(students);
```

```
  res.status(201).json({ message: 'Student created successfully!',
student: newStudent });
});
```

```
// 4. PUT /students/:id
app.put('/students/:id', (req, res) => {
  const studentId = parseInt(req.params.id);
  const { name, email, program } = req.body;
  const students = loadStudents();
  const index = students.findIndex(s => s.id === studentId);
```

```
  if (index === -1) {
    return res.status(404).json({ error: 'Student not found' });
  }
```

```
  if (email && (!typeof email === 'string' || !email.includes('@'))) {
    return res.status(400).json({ error: 'Validation Error: Email must
contain @' });
  }
```

```
  students[index] = {
    ...students[index],
    ...(name && { name: name.trim() }),
    ...(email && { email: email.trim() }),
    ...(program && { program: program.trim() })
  };
```

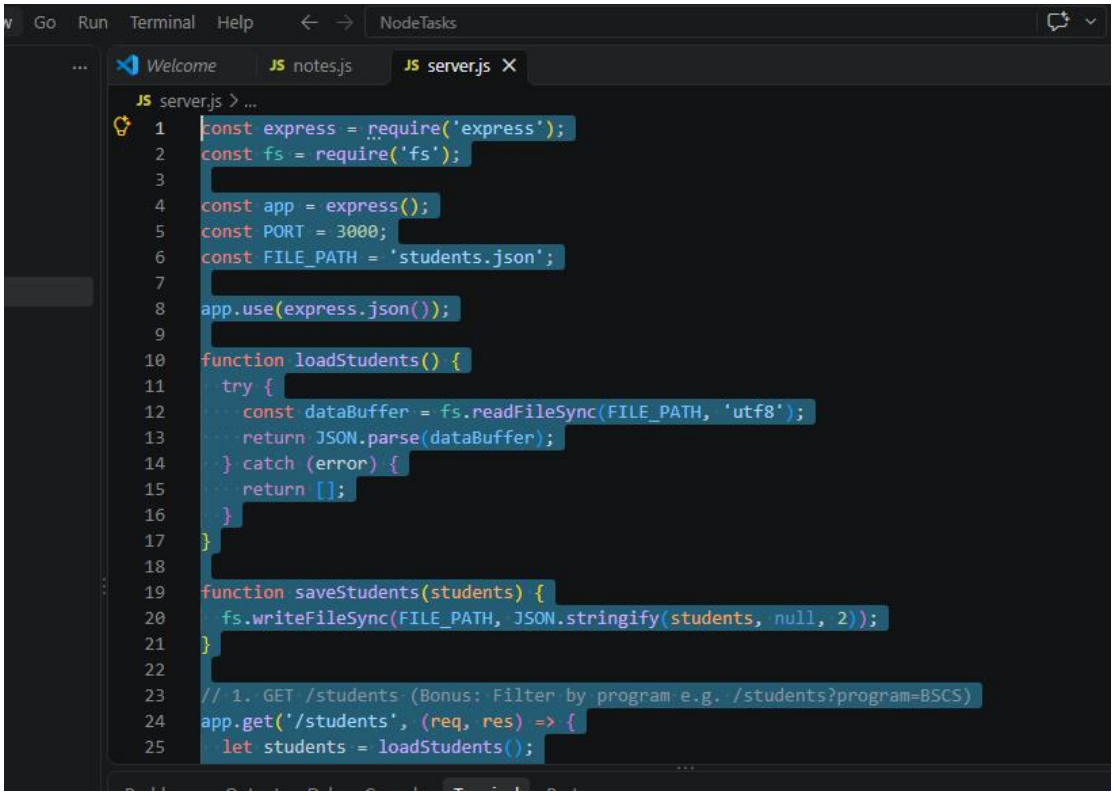
```
  saveStudents(students);
  res.status(200).json({ message: 'Student updated successfully!',
student: students[index] });
});
```

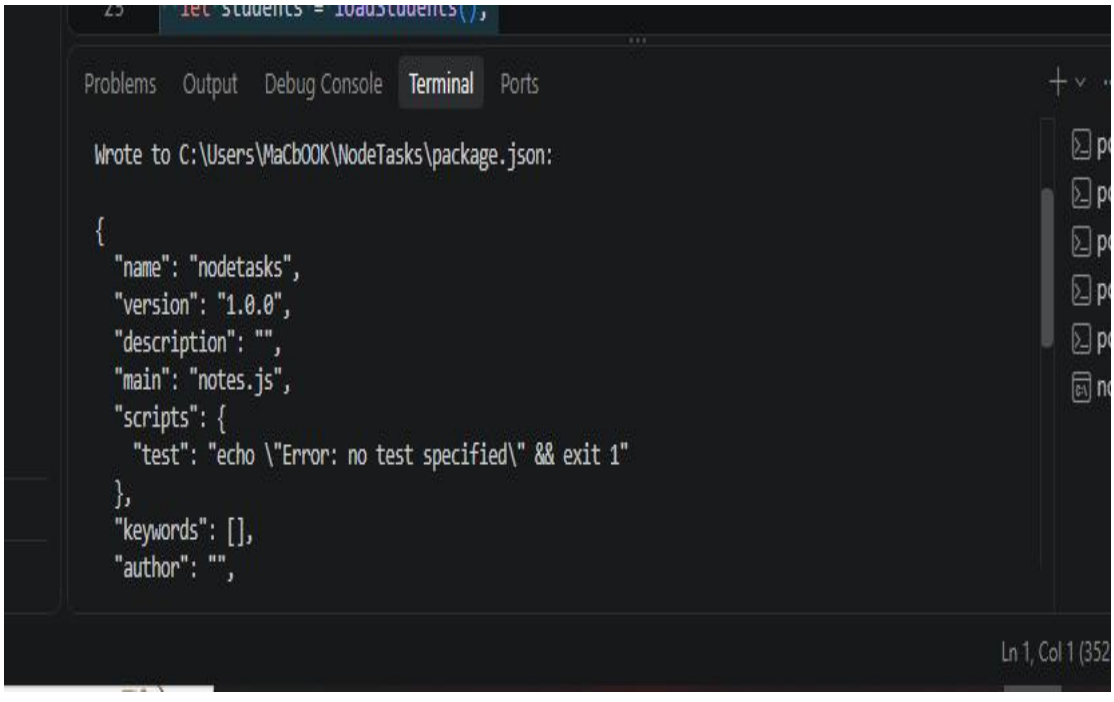
```
// 5. DELETE /students/:id
app.delete('/students/:id', (req, res) => {
  const studentId = parseInt(req.params.id);
  let students = loadStudents();
```

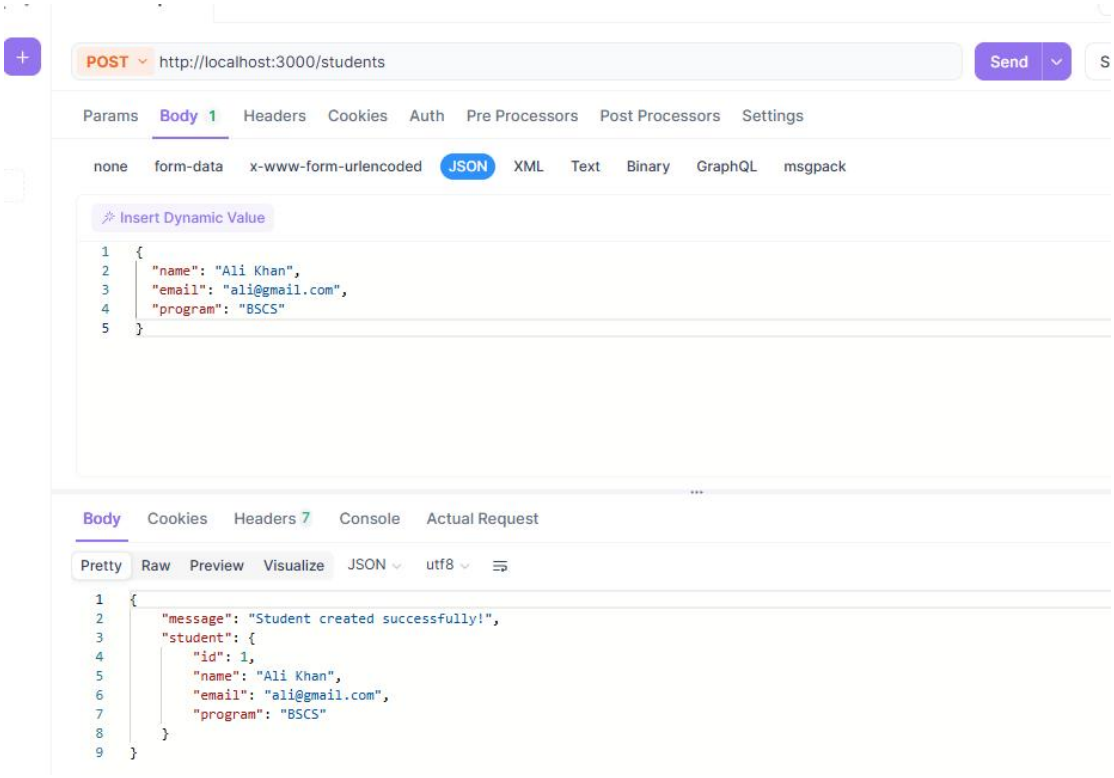
```
  if (!students.some(s => s.id === studentId)) {
    return res.status(404).json({ error: 'Student not found' });
  }
```

```
  students = students.filter(s => s.id !== studentId);
  saveStudents(students);
  res.status(200).json({ message: `Student with ID ${studentId} deleted
successfully.` });
});
```

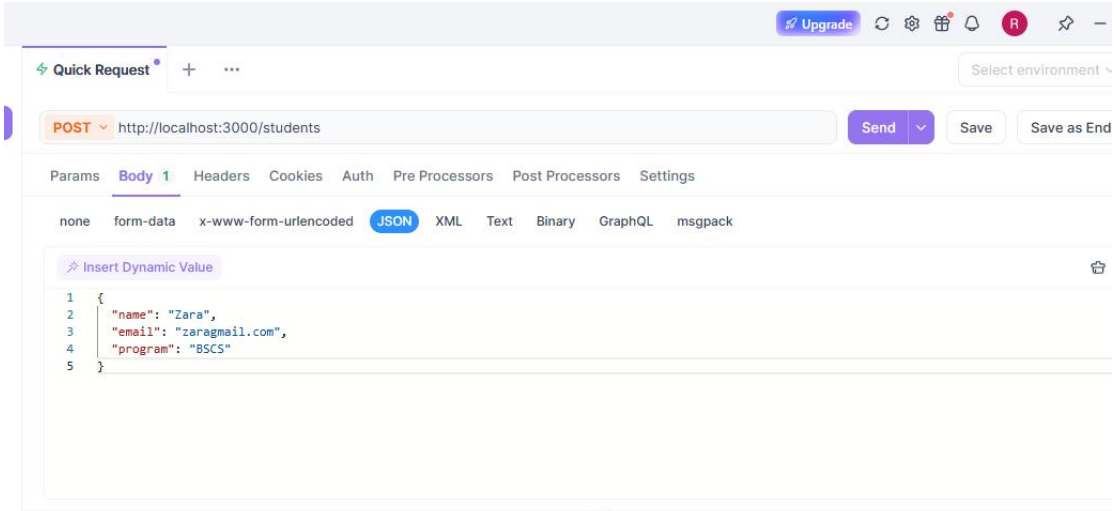
```
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

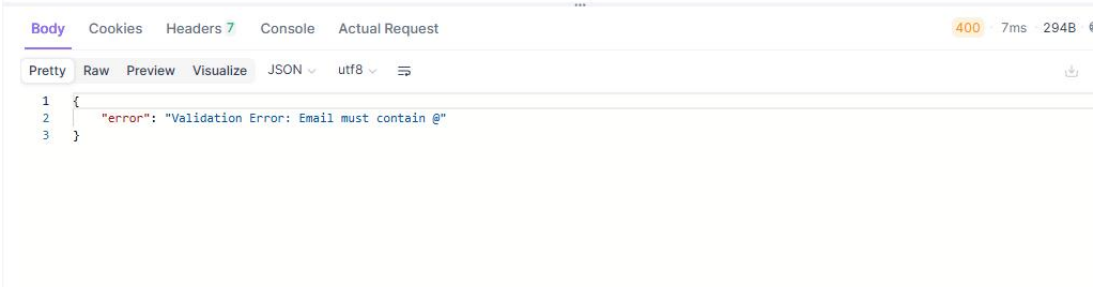

Go Run Terminal Help
NodeTasks
Welcome
notes.js
server.js
server.js > ...
1 const express = require('express');
2 const fs = require('fs');
3
4 const app = express();
5 const PORT = 3000;
6 const FILE_PATH = 'students.json';
7
8 app.use(express.json());
9
10 function loadStudents() {
11 try {
12 const dataBuffer = fs.readFileSync(FILE_PATH, 'utf8');
13 return JSON.parse(dataBuffer);
14 } catch (error) {
15 return [];
16 }
17 }
18
19 function saveStudents(students) {
20 fs.writeFileSync(FILE_PATH, JSON.stringify(students, null, 2));
21 }
22
23 // 1. GET /students (Bonus: Filter by program e.g. /students?program=BSCS)
24 app.get('/students', (req, res) => {
25 let students = loadStudents();

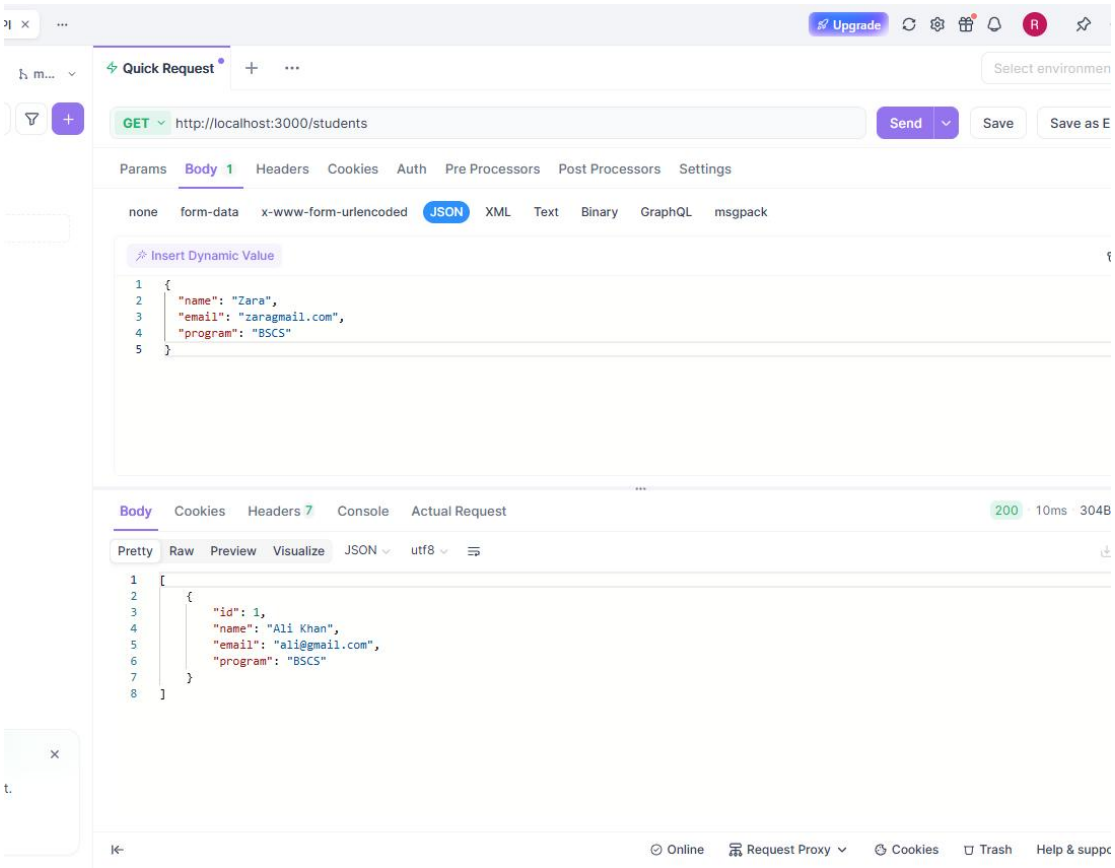

Problems
Output
Debug Console
Terminal
Ports
Wrote to C:\Users\MaCbOOK\NodeTasks\package.json:
{
"name": "nodetasks",
"version": "1.0.0",
"description": "",
"main": "notes.js",
"scripts": {
"test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
Ln 1, Col 1 (352

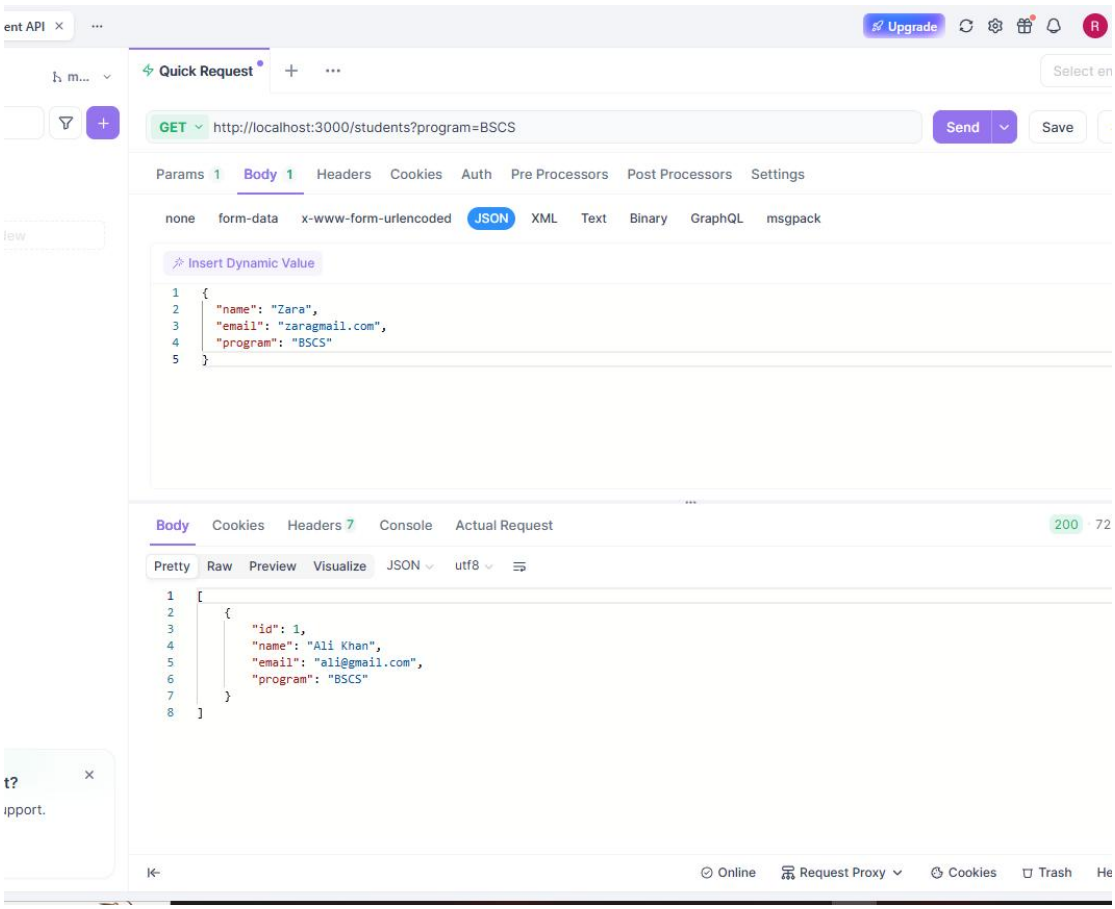

POST http://localhost:3000/students
Send
Params Body 1 Headers Cookies Auth Pre Processors Post Processors Settings
none form-data x-www-form-urlencoded JSON XML Text Binary GraphQL msgpack
Insert Dynamic Value
{
"name": "Ali Khan",
"email": "ali@gmail.com",
"program": "BSCS"
}

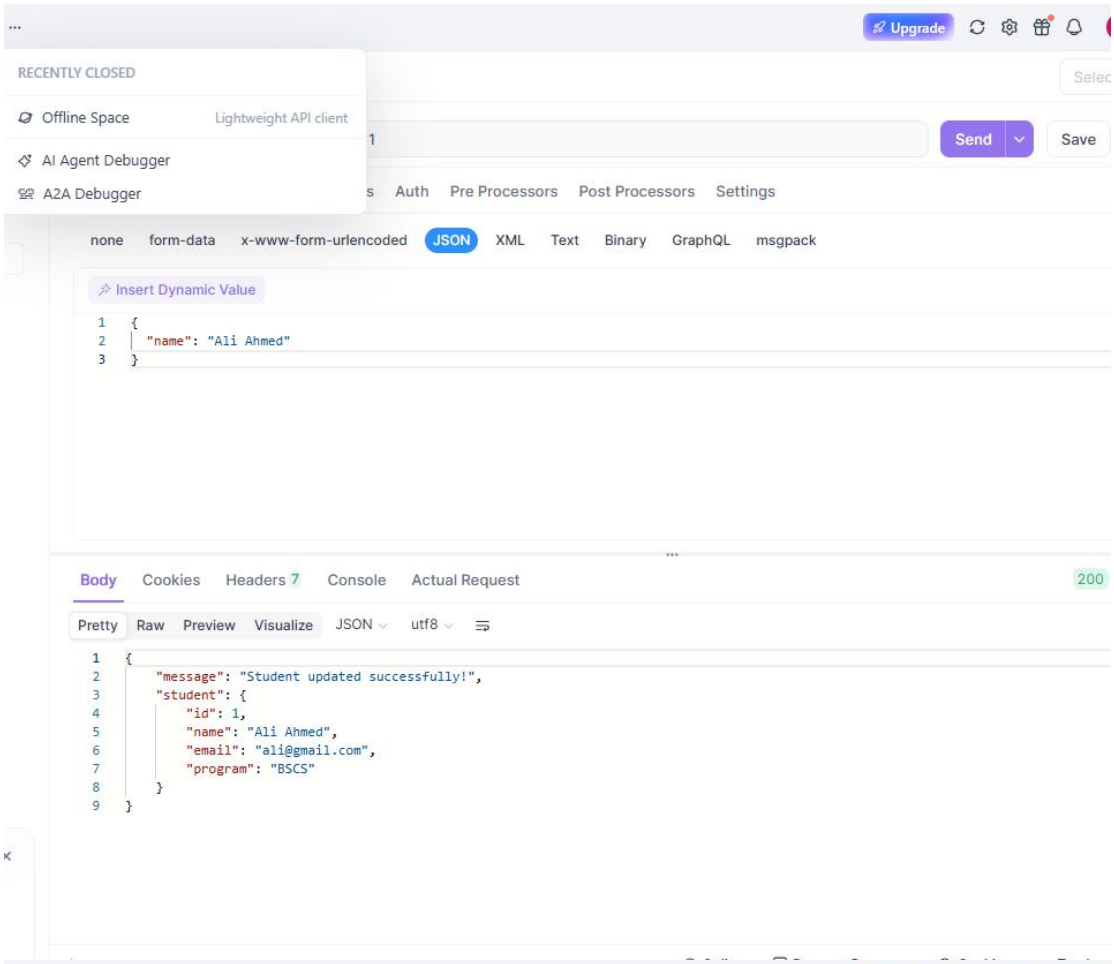
Body Cookies Headers 7 Console Actual Request
Pretty Raw Preview Visualize JSON utf8
{
"message": "Student created successfully!",
"student": {
"id": 1,
"name": "Ali Khan",
"email": "ali@gmail.com",
"program": "BSCS"
}
}

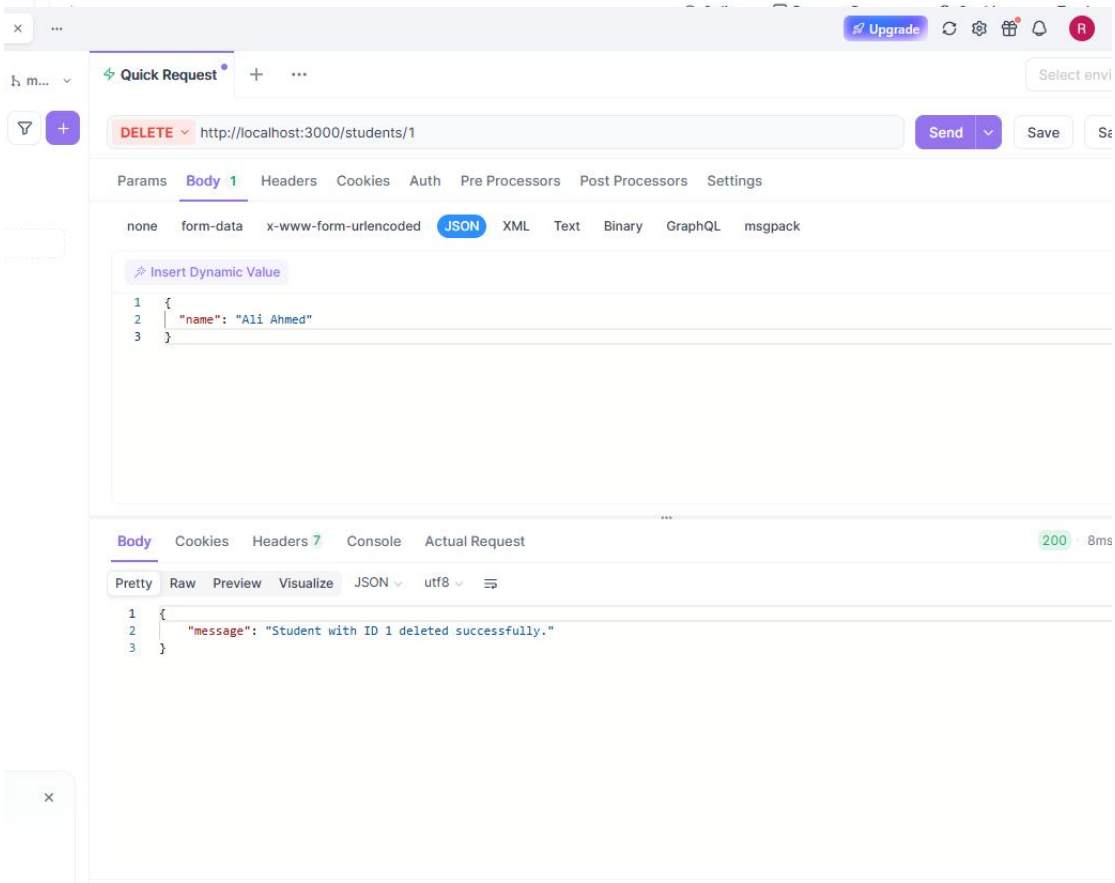

Upgrade
Quick Request
Select environment
POST http://localhost:3000/students
Send
Save
Save as End
Params Body 1 Headers Cookies Auth Pre Processors Post Processors Settings
none form-data x-www-form-urlencoded JSON XML Text Binary GraphQL msgpack
Insert Dynamic Value
{
"name": "Zara",
"email": "zaragmail.com",
"program": "BSCS"
}


Body Cookies Headers 7 Console Actual Request
400 7ms 294B
Pretty Raw Preview Visualize JSON utf8
{
"error": "Validation Error: Email must contain @"
}


Upgrade
Quick Request
GET http://localhost:3000/students
Send
Save
Params Body 1 Headers Cookies Auth Pre Processors Post Processors Settings
none form-data x-www-form-urlencoded JSON XML Text Binary GraphQL msgpack
Insert Dynamic Value
{
"name": "Zara",
"email": "zaragmail.com",
"program": "BSCS"
}
Body Cookies Headers 7 Console Actual Request
200 10ms 304B
Pretty Raw Preview Visualize JSON utf8
[
{
"id": 1,
"name": "Ali Khan",
"email": "ali@gmail.com",
"program": "BSCS"
}
]
Online Request Proxy Cookies Trash


Upgrade
Quick Request
GET http://localhost:3000/students?program=BSCS
Send
Save
Params 1 Body 1 Headers Cookies Auth Pre Processors Post Processors Settings
none form-data x-www-form-urlencoded JSON XML Text Binary GraphQL msgpack
Insert Dynamic Value
{
"name": "Zara",
"email": "zaragmail.com",
"program": "BSCS"
}
Body Cookies Headers 7 Console Actual Request
200 72
Pretty Raw Preview Visualize JSON utf8
[
{
"id": 1,
"name": "Ali Khan",
"email": "ali@gmail.com",
"program": "BSCS"
}
]
Online Request Proxy Cookies Trash


Upgrade
RECENTLY CLOSED
Offline Space
Lightweight API client
AI Agent Debugger
A2A Debugger
Send
Save
Auth
Pre Processors
Post Processors
Settings
none
form-data
x-www-form-urlencoded
JSON
XML
Text
Binary
GraphQL
msgpack
Insert Dynamic Value
{
"name": "Ali Ahmed"
}
Body
Cookies
Headers 7
Console
Actual Request
200
Pretty
Raw
Preview
Visualize
JSON
utf8
{
"message": "Student updated successfully!",
"student": {
"id": 1,
"name": "Ali Ahmed",
"email": "ali@gmail.com",
"program": "BSCS"
}
}


Upgrade
Quick Request
Select envi
DELETE
http://localhost:3000/students/1
Send
Save
Params
Body 1
Headers
Cookies
Auth
Pre Processors
Post Processors
Settings
none
form-data
x-www-form-urlencoded
JSON
XML
Text
Binary
GraphQL
msgpack
Insert Dynamic Value
{
"name": "Ali Ahmed"
}
Body
Cookies
Headers 7
Console
Actual Request
200
8ms
Pretty
Raw
Preview
Visualize
JSON
utf8
{
"message": "Student with ID 1 deleted successfully."
}